

普及组初赛模拟题（七）

一、单项选择题（共15题，每题2分，共计30分；每题有且仅有一个正确选项）

- 表达式 $(\text{int})\text{ceil}(3.7)$ 的结果为（ ）。
 - A. 4.0
 - B. 4
 - C. 3.0
 - D. 3
- 设一个有序的单链表有 n 个结点，现要求插入一个新节点后使得单链表依然保持有序，则该操作的时间复杂度为（ ）。
 - A. $\mathcal{O}(\log n)$
 - B. $\mathcal{O}(1)$
 - C. $\mathcal{O}(n^2)$
 - D. $\mathcal{O}(n)$
- Miku* 是一位调酒师，有强迫症的她希望调出水 and 酒精比例恰好为 $x : y$ 的酒精饮料，但是 *Miku* 的调酒技术十分的笨拙，她每次只能进行如下的操作：
 - 1 向杯子里加满水或者酒精
 - 2 将杯子里的液体倒出一半比如如果要调出水：酒精为 $5 : 3$ 的饮料，那么 *Miku* 可以进行如下的操作：
 - 1 加满水，目前为 $1 : 0$
 - 2 倒出一半液体，加满酒精，目前为 $1 : 1$
 - 3 倒出一半液体，加满酒精，目前为 $1 : 3$
 - 4 倒出一半液体，加满水，目前为 $5 : 3$这样 *Miku* 就得到了她想要的饮料，但是我们不难发现有些比例的饮料是 *Miku* 永远无法调制出来的，下列选项中的比例是 *Miku* 永远无法调制出来的为（ ）。
 - A. $9 : 7$
 - B. $21 : 11$
 - C. $5 : 7$
 - D. $3 : 29$
- 今天是仓鼠的生日，仓鼠给班里的同学们买来了一个长条蛋糕，为了分发的方便，仓鼠想要先将蛋糕沿着短边方向切好为若干段便于分发，但是仓鼠忘记了班里今天来了多少人，他只记得可能是 12 或者是 16，请问仓鼠最少要将蛋糕切多少刀才能使得任何情况下都存在一种分法使得每个人得到的蛋糕大小相同（ ）。

- A. 22
- B. 23
- C. 24
- D. 25

5. 使用 1×2 和 2×1 大小的多米诺骨牌填满 2×8 的方格的方案数为 ()。

- A. 21
- B. 22
- C. 33
- D. 34

6. 在算法竞赛中，我们经常遇到一些求解最优性的问题，比如经典的跳石头问题：

比赛在一条笔直的河道中进行，有 n 块岩石，现在可以移走 m 块岩石，使得相邻两块石头之间的距离尽可能长，请问进行了 m 次移动后相邻两块岩石之间距离最小值最大为多少？

对于这一类问题我们发现如果对于一个距离 k ，我们容易判断是否存在一种移走石头的方案能够使得其合法，我们就将一个最优性问题转换为了存在性问题。

这样对于求解这道题，我们为了得到较低的时间复杂度，我们可以考虑使用的算法是 ()。

- A. 递推法
- B. 递归法
- C. 二分法
- D. 倍增法

7. 在通过两个数组来模拟高精度乘法时，我们假定数组 a, b 分别表示两个乘数，那么对于两个数组单元 a_i, b_j ，其进行运算后最低位在答案数组 c 中的位置为 ()。

- A. $i + j$
- B. $i + j - 1$
- C. $i + j + 1$
- D. $i \times j$

8. 设无向图 G 中边的集合 $E = \{(a, b), (a, e), (a, c), (b, e), (e, d), (d, f), (f, c)\}$ ，则从顶点 a 出发进行深度优先遍历可以得到的一种顶点序列为 ()。

- A. $aedfcb$
- B. $acfebd$
- C. $aebcfd$
- D. $aedfbc$

9. 从一个 3 元素的集合中允许重复地有序选取 5 个元素有多少种不同的方式 ()。

- A. 81
- B. 243
- C. 27
- D. 9

10. 设某棵二叉树的中序遍历序列为 $ABCD$ ，前序遍历序列为 $CABD$ ，则后序遍历该二叉树得到的序列为 ()。

- A. $BADC$
- B. $BCDA$
- C. $CDAB$
- D. $CBDA$

11. RMQ 问题（区间最值查询）是算法竞赛中的经典问题，对于不带修改的 RMQ 问题，我们可以考虑使用 ST 表进行解决，我们定义 $st[i][j]$ 表示以 i 为起点，长度为 2^j 的区间中最大值为多少，则其查询区间最值的时间复杂度为多少并且这里用到了什么样的思想（）。

- A. $\mathcal{O}(\log n)$ ，二分
- B. $\mathcal{O}(1)$ ，二分
- C. $\mathcal{O}(\log n)$ ，倍增
- D. $\mathcal{O}(1)$ ，倍增

12. 在一个链上有 n 堆石子，每一堆石子的数目为 a_i ，你现在要进行 n 次合并操作，每次可以合并相邻的两堆石子，能获得新的一堆石子中的石子数量的得分，最大化得分。

对于这个问题，我们不难发现其为区间 DP 问题，我们考虑 $f(i, j)$ 表示将区间 $[i, j]$ 内的所有石子合并到一起的最大得分，那么下述状态转移方程正确的是（）。

- A. $f(i, j) = \max\{f(i, k) + f(k + 1, j) + \sum_{t=i}^j a_t\} (i \leq k < j)$
- B. $f(i, j) = \max\{f(i, k) + f(k, j) + \sum_{t=i}^j a_t\} (i \leq k \leq j)$
- C. $f(i, j) = \max\{f(i, k) + f(k - 1, j) + \sum_{t=i}^j a_t\} (i < k < j)$
- D. $f(i, j) = \sum_{i=k}^j \max\{f(i, k), f(k, j), \sum_{t=i}^j a_t\}$

13. 在内存存储器中每个存储单元都被赋予一个唯一的序号，称为（）。

- A. 序号
- B. 地址
- C. 单元
- D. 下标

14. 天干地支源自于中国古代对于天象的观测，简化后的天干地支分别为“甲、乙、丙、丁、戊、己、庚、辛、壬、癸”和“子、丑、寅、卯、辰、巳、午、未、申、酉、戌、亥”，每经过一年，天干和地支都向后移动一位，循环往复，则两个辛亥年之间间隔多少年（）。

- A. 60
- B. 120
- C. 22
- D. 100

15. 一个序列逆序对的数目对于描述一个序列的性质有着重要作用，下列能统计序列逆序对数目的算法有（）。

- A. 快速排序
- B. 选择排序
- C. 归并排序
- D. 后缀排序

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填 T，错误填 F；除特殊说明外，判断题 1.5 分，选择题 3 分，共计 40 分）

注意：判断题正确填 T，错误填 F。

阅读下面程序，完成第 16 ~ 21 题。

```
1  #include<bits/stdc++.h>
2
3  using namespace std;
4
5  int n, k;
6  vector < int > a;
7
8  int main() {
9      cin >> n;
10     for (int i = 1, x; i <= n; ++i) {
11         cin >> x;
12         a.push_back(x);
13     }
14     sort(a.begin(), a.end());
15     cin >> k;
16     cout << a[lower_bound(a.begin(), a.end(), k) - a.begin()] << endl;
17     return 0;
18 }
```

保证 k 小于 $\max\{a_i\} - 1$ 。

16. 上述代码可以求出序列 a 中大于 k 的数中的最小的数。[]
17. 将 `a.push_back(x);` 改为 `a[i]=x;`，程序输出结果一定不变。[]
18. 若将 `sort(a.begin(), a.end());` 删去程序有可能会运行出错。[]
19. 此程序的时间复杂度为 $O(\log n)$ 。[]
20. 若输入为

```
5
1 2 3 4 5
2
```

则输出为（ ）。

- A. 2
- B. 3
- C. 4

◦ D. 5

21. 若将 `cout << a[lower_bound(a.begin(), a.end(), k) - a.begin()] << endl;` 改为 `cout << a[upper_bound(a.begin(), a.end(), k) - a.begin()] << endl;` , 则下列输入中答案不会发生改变的为 () 。

◦ A.

```
5
5 4 3 2 1
0
```

◦ B.

```
5
5 4 3 2 1
1
```

◦ C.

```
5
5 4 3 2 1
2
```

◦ D.

```
5
5 4 3 2 1
3
```

阅读下面程序，完成第 22 ~ 27 题。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  const int maxn = 500010;
4  string str;
5  int pos[maxn];
6  long long solve(int m, int n) {
7      long long ans = 0;
8      pos[0] = 0, pos[m + 1] = n + 1;
9      for (int i = 1; i <= m; i++) {
10         int left = pos[i] - pos[i - 1] - 1;
11         int right = pos[i + 1] - pos[i] - 1;
12         if (left >= 2) {
13             ans += left - 1;
14         }
15         if (right >= 2) {
16             ans += right - 1;
17         }
18         if (left > 0 && right > 0) {
19             ans += (long long)left * right;
20         }
21     }
22     return ans;
23 }
24 int main() {
25     int n;
26     cin >> n;
27     cin >> str;
28     int m = 0;
29     for (int i = 0; i < n; i++) {
30         if (str[i] == 'G') {
31             pos[++m] = i + 1;
32         }
33     }
34     long long ans = solve(m, n);
35     m = 0;
36     for (int i = 0; i < n; i++) {
37         if (str[i] == 'H') {
38             pos[++m] = i + 1;
39         }
40     }
41     ans += solve(m, n);
42     cout << ans << endl;
43     return 0;
44 }

```

保证输入的字符串中只包含 H 和 G 。

22. 第 31 行和第 38 行中 pos 数组内分别记录了每个字母 G 和 H 的索引位置（经过 +1 处理后的结果）。[]

23. 删除第 8 行代码后，程序输出不会改变。[]
24. 将第 29 ~ 32 行代码与第 36 ~ 40 行代码交换位置，程序输出会改变。[]
25. 程序的含义是求解字符串 *str* 中长度大于等于 3 的子串中，有且仅有一个字母与其他字母不同的子串个数。[]
26. 若输入

5
GHGHG

则输出为（ ）。

- A. 1
 - B. 2
 - C. 3
 - D. 4
27. (4分) 选出下面说法正确的是（ ）。
- A. 第 9 行代码是枚举满足条件的子串中的中间字母位置。
 - B. 第 10, 11 行代码中 `left` 和 `right` 是指枚举的字母位置左右两边相同字母的个数。
 - C. 第 13 行代码中 `left - 1` 是指：以 `pos[i]` 作为子串的右端点且该子串满足条件的个数。
 - D. 第 19 行代码中 `long long` 可以删除，程序输出总是不会改变。

阅读下面程序，完成第 28 ~ 33 题。

```

1  #include<bits/stdc++.h>
2
3  using namespace std;
4
5  struct Bigint: vector < int > {
6      Bigint(int x = 0) {
7          push_back(x);
8          check();
9      }
10     Bigint & check() {
11         while (!empty() && !back()) pop_back();
12         if (empty()) return *this;
13         for (int i = 1; i < size(); ++i) {
14             ( * this)[i] += ( * this)[i - 1] / 10;
15             ( * this)[i - 1] %= 10;
16         }
17         while (back() >= 10) {
18             push_back(back() / 10);
19             ( * this)[size() - 2] %= 10;
20         }
21         return *this;
22     }
23 };
24 istream & operator >> (istream & is, Bigint & x) {
25     string s;
26     is >> s;
27     x.clear();
28     for (int i = s.size() - 1; i >= 0; --i) x.push_back(s[i] - '0');
29     return is;
30 }
31 ostream & operator << (ostream & os,
32     const Bigint & x) {
33     if (x.empty()) os << 0;
34     for (int i = x.size() - 1; i >= 0; --i) os << x[i];
35     return os;
36 }
37 Bigint & operator += (Bigint & a,
38     const Bigint & b) {
39     if (a.size() < b.size()) a.resize(b.size());
40     for (int i = 0; i != b.size(); ++i) a[i] += b[i];
41     return a.check();
42 }
43 Bigint & operator + (Bigint a,
44     const Bigint & b) {
45     return a += b;
46 }
47
48 int main() {
49     Bigint a, b;
50     cin >> a >> b;

```



```

51 |     cout << a + b;
52 |     return 0;
53 | }

```

28. 上述代码在模拟高精度加法。[]

29. 若将 `cout<<a+b;` 改为 `cout<<a+=b;` , 则输出一定不会发生变化。[]

30. 该算法的时间复杂度为()

- A. $\mathcal{O}(1)$
- B. $\mathcal{O}(\log n)$
- C. $\mathcal{O}(n)$
- D. $\mathcal{O}(n^2)$

31. 当程序输入为

```

567483256438765782345
436758623478543432

```

程序的输出为 () 。

- A. 567920015062244325777
- B. 567920015062244325776
- C. 667920015062244325777
- D. 667920015062244325776

32. 当输入的 $a < b$ 时, 下列说法正确的是 () 。

- A. 程序能够正确的输出答案但是有可能运行出错
- B. 程序不能够正确的输出答案并且会运行出错
- C. 程序能够正确的输出答案并且运行不会出错
- D. 程序不能够正确的输出答案但不会运行出错

33. 如果要重载上述代码类型的小于号 `<` , 则下列写法正确的是 () 。

- A.

```

1 | bool operator < (const Bigint & a,
2 |   const Bigint & b) {
3 |     if (a.size() != b.size()) return a.size() < b.size();
4 |     for (int i = a.size() - 1; i >= 0; --i)
5 |         if (a[i] != b[i]) return a[i] < b[i];
6 |     return 0;
7 | }

```

- B.

```

1 | bool operator < (const Bigint & a,
2 |     const Bigint & b) {
3 |     for (int i = a.size() - 1; i >= 0; --i)
4 |         if (a[i] != b[i]) return a[i] < b[i];
5 |     return 0;
6 | }

```

◦ C.

```

1 | bool operator < (const Bigint & a,
2 |     const Bigint & b) {
3 |     if (a.size() != b.size()) return a.size() < b.size();
4 |     for (int i = a.size(); i >= 0; --i)
5 |         if (a[i] != b[i]) return a[i] < b[i];
6 |     return 0;
7 | }

```

◦ D.

```

1 | bool operator < (const Bigint & a,
2 |     const Bigint & b) {
3 |     for (int i = a.size(); i >= 0; --i)
4 |         if (a[i] != b[i]) return a[i] < b[i];
5 |     return 0;
6 | }

```

三、完善程序（单选题，每小题3分，共计30分）

阅读下面题目，完成第 34 ~ 38 题。

题目描述

一共有 n 个陷阱，编号从 1 到 n ，你要按顺序一个个走过，第 i 个陷阱会对你造成 a_i 点伤害。

现在你有 k 次机会，对于每一次机会你可以选择跳过一个陷阱，如果你跳过一个陷阱，那么它就不会对你造成伤害，但是剩下的陷阱造成的伤害都会 $+1$ 。

如果你被允许跳过 k 个陷阱，求解你能受到的最少伤害。

输入格式

第一行为一个正整数 T 表示数据组数。

对于每一组数据，第一行为两个正整数 n, k 分别为陷阱数目和 k 机会的数目。

第二行为 n 个正整数 a_i 表示第 i 个陷阱造成的伤害。

输出格式

对于每一组数据输出仅一行，表示最少受到的伤害。

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  const int N = 200020;
6
7  long long a[N];
8
9  int main() {
10     int t;
11     int n, k;
12     int i;
13     long long ans;
14
15     cin >> t;
16     while (t--) {
17         cin >> n >> k;
18         for (i = 1; i <= n; i++) ①
19         for (i = 1, ans = 0; i <= n; i++) {
20             ②
21             a[i] += i;
22         }
23         ③
24         for (i = 1; i <= ④; i++)
25             ⑤
26         cout << ans << endl;
27     }
28
29     return 0;
30 }
```

1. ①处应填()

- A. `cin >> a[i];`
- B. `cin << a[i];`
- C. `cin >> a;`
- D. `cin << a;`

2. ②处应填()

- A. `ans += a[i];`
- B. `ans -= a[i];`
- C. `ans *= a[i];`

- D. `ans /= a[i];`

3. ③处应填()

- A. `sort(a + 1, a + n, greater < int > ());`
- B. `sort(a + 1, a + n + 1, greater < int > ());`
- C. `sort(a + 1, a + n);`
- D. `sort(a + 1, a + n + 1);`

4. ④处应填()

- A. `k`
- B. `k-1`
- C. `n`
- D. `n-1`

5. ⑤处应填()

- A. `ans += n - i + 1;`
- B. `ans += n - i - a[i];`
- C. `ans += n - i + 1 - a[i];`
- D. `ans += n + 1 - a[i];`

阅读下面题目，完成第 39 ~ 43 题。

题目描述

你有两个正整数 n 和 x ，你需要对 x 进行若干次操作使得 x 在十进制下的数位长度大于 n 。

每一次操作，你可以选择 x 在十进制下每位中出现过的数 y ($0 \leq y < 10$)，将 x 变为 $x \times y$ 。

请问最少的操作次数为多少，如果无论如何都不能满足条件则输出 -1 。

输入格式

输入仅一行，两个正整数 n, x 。

输出格式

输出仅一行，表示最少的操作次数，不满足条件输出 -1 。

注：其中 *map* 能记录一个数是否出现过，本题仅用来优化，不设考点。

```

1  #include<bits/stdc++.h>
2
3  using namespace std;
4
5  #define ll long long
6
7  struct node {
8      ll val, stp;
9  };
10
11  ll n, x;
12  queue < node > q;
13  map < ll, bool > vis;
14
15  ll CalcBit(ll x) {
16      int t = 0;
17      while (x) {
18          ++t;
19          x /= 10;
20      }
21      return t;
22  }
23  int main() {
24      cin >> n >> x;
25      __ (1) __
26      ll ans = __ (2) __;
27      while (!q.empty()) {
28          node s = q.front();
29          q.pop();
30          ll res = s.val;
31          if (__ (3) __) {
32              ans = min(ans, s.stp);
33              break;
34          }
35          while (res) {
36              ll t = res % 10;
37              res /= 10;
38              if (__ (4) __ && CalcBit(s.val * t) <= n) {
39                  __ (5) __;
40              }
41          }
42      }
43      if (ans == 1000) cout << -1 << endl;
44      else cout << ans << endl;
45      return 0;
46  }

```

39. 上述程序__ (1) __中应该填写 ()

- A. q.push({x,0});

- B. `q.push({x,1});`
- C. `q.push(x,0);`
- D. `q.push(x,1);`

40. 上述程序__(2)__中应该填写 ()

- A. 1
- B. 5
- C. 10
- D. 1000

41. 上述程序__(3)__中应该填写 ()

- A. `CalcBit(res)>n`
- B. `CalcBit(res)>=n`
- C. `CalcBit(res)<n`
- D. `CalcBit(res)<=n`

42. 上述程序__(4)__中应该填写 ()

- A. `vis[s.val*t]`
- B. `!vis[s.val]`
- C. `!vis[s.val*t]`
- D. `vis[s.val]`

43. 上述程序__(5)__中应该填写 ()

- A. `vis[s.val * t] = true, q.push({ s.val * t, s.stp + 1 });`
- B. `vis[s.val * t] = true, q.push({ t, s.stp + 1 });`
- C. `vis[s.val * t] = false, q.push({ s.val * t, s.stp + 1 });`
- D. `vis[s.val * t] = false, q.push({ t, s.stp + 1 });`